

Fixed-Window QK Relational Architecture

Python Quickstart

Minimal runnable implementation of linear ordinary reading and optional strict hops

Contact: rickeyernest1990@gmail.com
Publication date: 14 September 2026

Purpose

This document contains one compact, self-contained PyTorch program intended to get a reader started with the fixed-window companion architecture. The defaults reproduce the tested scaled model shape: two layers, width $d = 96$, fixed ordinary window $W = d = 96$, byte vocabulary 256, no learned value projection in the QK reader, and separate final-block Q_h, K_h projections for optional strict no-self hopping.

The important change from the earlier full-pair starter is that ordinary reading does *not* construct a global $N \times N$ A_0 matrix. It evaluates only the legal local QK relationships in small blocks. For a fixed model,

$$W = cd, \quad A_0^{(W)} \text{ work} = O(NWd) = O(N)$$

with respect to sequence length N . The tested default is $c = 1$.

The hop path remains deliberately separate. When `extra_hops=0`, its projections are dormant. When hops are requested, the current quickstart constructs the strict full-history A_h , so that optional mode is still quadratic in N .

Core mathematics

Let $H \in \mathbb{R}^{N \times d}$ be the normalized hidden state. Ordinary projections are

$$Q_0 = HW_{Q_0}, \quad K_0 = HW_{K_0}.$$

For query position i , only

$$\mathcal{J}_W(i) = \{j : j \leq i, i - j < W\}$$

is legal. The distance prior is

$$D(r) = \frac{0.02}{1+r} + \frac{0.98}{(1+r)^2}.$$

The local ordinary weights are

$$A_{0,ij}^{(W)} \propto \exp\left(\frac{q_i k_j^T}{\sqrt{d}} + \log D(i-j)\right), \quad j \in \mathcal{J}_W(i).$$

At ordinary depth $k = 0$, the transmitted content is the raw normalized state:

$$Z_0 = A_0^{(W)} H.$$

There is no learned V projection.

The optional hop geometry is

$$Q_h = HW_{Q_h}, \quad K_h = HW_{K_h},$$

$$A_h = \text{softmax}\left(\frac{Q_h K_h^T}{\sqrt{d}} + M_{<}\right),$$

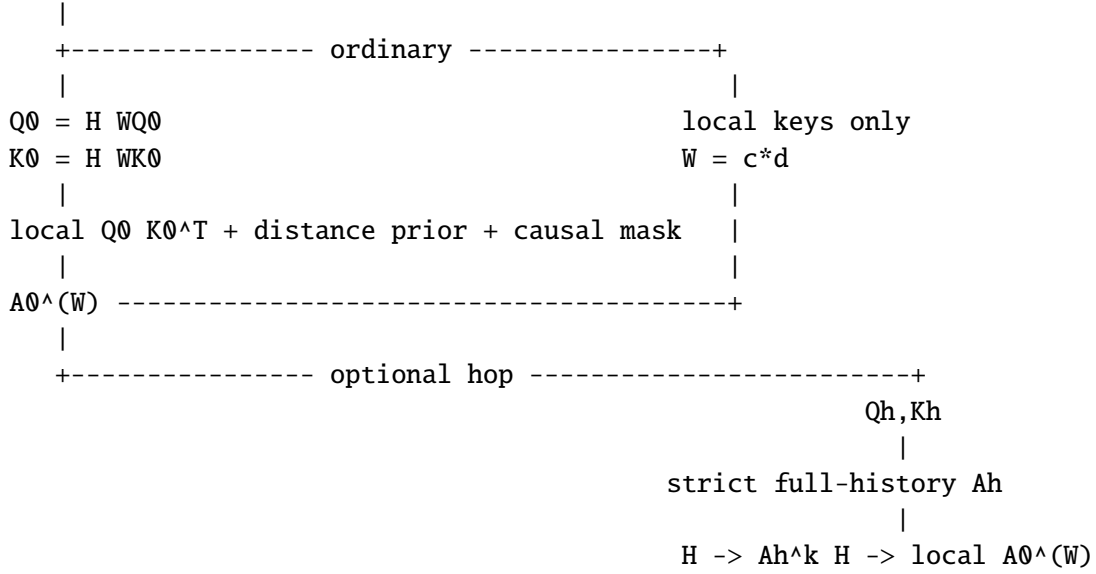
where $M_{<}$ forbids self transitions for every row after row zero. Requested depth is

$$Z_k = A_0^{(W)} A_h^k H.$$

The code implements this exact order by first forming $z = A_h^k H$ and then applying the local ordinary read to z using the Q_0, K_0 weights computed from H .

Architecture at a glance

normalized H



$k = 0$: cheap local ordinary path only

$k > 0$: strict hop path is activated; current A_h is still dense

Default tested-size configuration

Setting	Default in this starter
Layers	2
Model width d	96
Window rule	$W = cd, c = 1$
Ordinary window W	96
Vocabulary	256 bytes
Block 1 FF width	$5d = 480$
Final block FF width	$4d = 384$
Hop projections	final block only
Stored parameters	271,296
Ordinary training mode	<code>extra_hops=0</code>

The separate hop geometry adds $2d^2 = 18,432$ learned parameters at $d = 96$; those parameters are already included in the 271,296 total. They are stored with the model but do not produce hop activations during ordinary $k = 0$ operation.

Install and run

Install the only required packages:

```
python -m pip install torch numpy
```

Run the architecture checks:

```
python qkra_fixed_window_minimal.py --smoke
```

The copy supplied with this document was executed before publication. Its smoke test passed the 271,296-parameter check, exact local-versus-dense-window equivalence check, finite normal and hopped forward passes, strict no-self hop diagonal check, and dormant-versus-active hop-gradient check. The observed local-versus-dense maximum absolute difference in that run was 0.0 in float32.

For a small next-byte training run on any local byte file:

```
python qkra_fixed_window_minimal.py \  
--train-byte-file version9.txt \  
--steps 1000 --context 512 --batch 4 --seed 11
```

The starter uses AdamW at 5×10^{-4} , weight decay 0.01 and gradient clipping at 1.0. A two-step training smoke run was also executed successfully before this document was generated. Normal language training intentionally uses `extra_hops=0`; a relational task must explicitly request hops in the forward pass.

What to edit first

The simplest architectural knob is `window_multiplier`. The model computes

$$W = \text{round}(c d).$$

The reported companion experiment used $c = 1$. A larger fixed c gives a wider ordinary field while remaining linear in N for a fixed model. Changing d , feed-forward widths, training data or hop supervision creates a new experiment and should not be described as reproducing the reported benchmark without a fresh matched control.

The `block_size` argument changes implementation chunking only. It does not change the legal relationship window or model mathematics. This starter favors transparent explicit matrix multiplications inside each local block. The timing study used an additionally optimized local kernel, so this compact file is a fidelity/reference implementation rather than a promise of maximum hardware speed.

Important limitation

Only the ordinary field has been made fixed-window here. At $k = 0$, the model uses at most NW ordinary relationships and never constructs an $N \times N$ ordinary score matrix. At $k > 0$, the current `strict_hop_matrix` still constructs a dense $N \times N$ A_h . Therefore the correct description is:

linear ordinary reading + optional quadratic relational hopping.

Making A_h sparse or indexed without destroying its learned transition accuracy remains separate research work.

Complete code: `qkra_fixed_window_minimal.py`

```

1  """Minimal runnable PyTorch starter for the fixed-window QK Relational Architecture.
2
3  Companion architecture (September 2026):
4  - 2 layers, default d_model=96, byte vocabulary=256
5  - ordinary QK field uses a fixed causal window W = c*d (default c=1, W=96)
6  - ordinary reading is headless and has no learned value projection
7  - fixed distance prior D(r)=0.02/(1+r)+0.98/(1+r)^2
8  - final block owns separate Qh/Kh projections for optional strict no-self hops
9  - requested extra hops compute A0^(W) @ Ah^k @ H
10 - default FF widths are 5d in block 1 and 4d in block 2
11 - default tested-size parameter count: 271,296
12
13 Ordinary k=0 operation never constructs an N x N A0 matrix. The local reader
14 uses small contiguous query/key blocks, so ordinary relationship work is O(N*W*d).
15 When extra_hops > 0, the current Ah implementation is still dense/full-history and
16 therefore quadratic in sequence length. This file prioritizes clarity and fidelity
17 rather than reproducing every optimization used in the timing benchmark.
18
19 Documentation/explanatory text: CC BY-NC 4.0.
20 Source code: PolyForm Noncommercial License 1.0.0.
21 Contact: rickeyernest1990@gmail.com
22 """
23
24 import argparse
25 import math
26 from pathlib import Path
27
28 import numpy as np
29 import torch
30 from torch import nn
31 from torch.nn import functional as F
32
33
34 DECAY_A = 0.02
35 DECAY_B = 0.98
36
37
38 def local_qk_read(values, q, k, window, block_size=64):
39     """Exact fixed-window causal QK read without an N x N ordinary score matrix.
```

```

40
41 Args:
42     values: state to transmit, [B, N, d]. This is raw H at k=0, or  $Ah^k H$ .
43     q, k: ordinary Q/K projections from the same normalized H, [B, N, d].
44     window: maximum number of causal positions visible to each query.
45     block_size: small query block used only for implementation efficiency.
46
47 Returns:
48     [B, N, d] local relational read.
49     """
50     B, N, d = values.shape
51     W = min(int(window), N)
52     pieces = []
53
54     for start in range(0, N, block_size):
55         end = min(N, start + block_size)
56         key_start = max(0, start - W + 1)
57         key_end = end
58
59         qb = q[:, start:end, :]
60         kb = k[:, key_start:key_end, :]
61         vb = values[:, key_start:key_end, :]
62
63         scores = (qb @ kb.transpose(-2, -1)) / math.sqrt(d)
64
65         qi = torch.arange(start, end, device=values.device)[:, None]
66         kj = torch.arange(key_start, key_end, device=values.device)[None, :]
67         delta = qi - kj
68         valid = (delta >= 0) & (delta < W)
69
70         r = delta.clamp_min(0).to(values.dtype)
71         log_prior = torch.log(
72             DECAPY_A / (1.0 + r) + DECAPY_B / (1.0 + r).square()
73         )
74
75         scores = scores + log_prior.unsqueeze(0)
76         scores = scores.masked_fill(~valid.unsqueeze(0), -torch.inf)
77         weights = scores.softmax(dim=-1)
78         pieces.append(weights @ vb)
79
80     return torch.cat(pieces, dim=1)
81
82
83 def strict_hop_matrix(h, qh, kh):
84     """Build the current full-history strict hop field  $A_h$ .
85
86     Rows  $i > 0$  may attend only to  $j < i$ . Row zero receives one fallback self entry so
87     softmax has a legal element. No later row may self-loop.
88     """
89     B, N, d = h.shape
90     scores = (qh @ kh.transpose(-2, -1)) / math.sqrt(d)
91
92     pos = torch.arange(N, device=h.device)
93     strict = pos[:, None] > pos[None, :]
94     strict = strict.clone()
95     strict[0, 0] = True
96
97     return scores.masked_fill(~strict.unsqueeze(0), -torch.inf).softmax(dim=-1)
98
99
100 def dense_local_reference(values, q, k, window):
101     """Dense N x N reference used only by the smoke test, not normal operation."""
102     B, N, d = values.shape
103     W = min(int(window), N)
104     pos = torch.arange(N, device=values.device)
105     delta = pos[:, None] - pos[None, :]
106     valid = (delta >= 0) & (delta < W)
107     r = delta.clamp_min(0).to(values.dtype)
108
109     scores = (q @ k.transpose(-2, -1)) / math.sqrt(d)
110     scores = scores + torch.log(
111         DECAPY_A / (1.0 + r) + DECAPY_B / (1.0 + r).square()
112     )
113     weights = scores.masked_fill(~valid.unsqueeze(0), -torch.inf).softmax(dim=-1)
114     return weights @ values, weights
115
116
117 class FixedWindowQKBlock(nn.Module):
118     def __init__(self, d_model, ff_dim, window, has_hop=False, block_size=64):
119         super().__init__()
120         self.window = int(window)
121         self.has_hop = bool(has_hop)

```

```

122     self.block_size = int(block_size)
123
124     self.ln1 = nn.LayerNorm(d_model)
125     self.ln2 = nn.LayerNorm(d_model)
126
127     self.q0 = nn.Linear(d_model, d_model, bias=False)
128     self.k0 = nn.Linear(d_model, d_model, bias=False)
129
130     if self.has_hop:
131         self.qh = nn.Linear(d_model, d_model, bias=False)
132         self.kh = nn.Linear(d_model, d_model, bias=False)
133
134     self.ff = nn.Sequential(
135         nn.Linear(d_model, ff_dim, bias=False),
136         nn.ReLU(),
137         nn.Linear(ff_dim, d_model, bias=False),
138     )
139
140     def forward(self, x, extra_hops=0):
141         h = self.ln1(x)
142         q0, k0 = self.q0(h), self.k0(h)
143
144         if extra_hops > 0:
145             if not self.has_hop:
146                 raise ValueError("extra_hops may only be used in the hop-enabled final block")
147
148             qh, kh = self.qh(h), self.kh(h)
149             Ah = strict_hop_matrix(h, qh, kh)
150
151             # Exact order:  $A0^{(W)} Ah^k H$ .
152             z = h
153             for _ in range(int(extra_hops)):
154                 z = Ah @ z
155
156             out = local_qk_read(z, q0, k0, self.window, self.block_size)
157         else:
158             # Ordinary mode: qh, kh and Ah are completely dormant.
159             out = local_qk_read(h, q0, k0, self.window, self.block_size)
160
161         x = x + out
162         return x + self.ff(self.ln2(x))
163
164
165 class FixedWindowQKLanguageModel(nn.Module):
166     def __init__(
167         self,
168         vocab_size=256,
169         d_model=96,
170         window_multiplier=1.0,
171         max_context=8192,
172         block_size=64,
173     ):
174         super().__init__()
175         self.vocab_size = int(vocab_size)
176         self.d_model = int(d_model)
177         self.window_multiplier = float(window_multiplier)
178         self.window = max(1, int(round(self.window_multiplier * self.d_model)))
179
180         self.embed = nn.Embedding(self.vocab_size, self.d_model)
181         self.blocks = nn.ModuleList([
182             FixedWindowQKBlock(
183                 self.d_model,
184                 ff_dim=5 * self.d_model,
185                 window=self.window,
186                 has_hop=False,
187                 block_size=block_size,
188             ),
189             FixedWindowQKBlock(
190                 self.d_model,
191                 ff_dim=4 * self.d_model,
192                 window=self.window,
193                 has_hop=True,
194                 block_size=block_size,
195             ),
196         ])
197         self.norm = nn.LayerNorm(self.d_model)
198         self.head = nn.Linear(self.d_model, self.vocab_size, bias=False)
199
200         pos = torch.arange(max_context)[: , None].float()
201         div = torch.exp(
202             torch.arange(0, self.d_model, 2).float()
203             * (-math.log(10000.0) / self.d_model)

```

```

204         )
205         pe = torch.zeros(max_context, self.d_model)
206         pe[:, 0::2] = torch.sin(pos * div)
207         pe[:, 1::2] = torch.cos(pos * div)
208         self.register_buffer("position", pe, persistent=False)
209
210         self.apply(self._init_weights)
211
212     @staticmethod
213     def _init_weights(module):
214         if isinstance(module, (nn.Linear, nn.Embedding)):
215             nn.init.normal_(module.weight, std=0.02)
216
217     def forward(self, tokens, extra_hops=0):
218         if tokens.shape[1] > self.position.shape[0]:
219             raise ValueError("sequence is longer than max_context")
220
221         h = self.embed(tokens) * math.sqrt(self.d_model)
222         h = h + self.position[: tokens.shape[1]]
223         h = self.blocks[0](h, extra_hops=0)
224         h = self.blocks[1](h, extra_hops=extra_hops)
225         return self.head(self.norm(h))
226
227
228     def parameter_count(model):
229         return sum(p.numel() for p in model.parameters())
230
231
232     def make_byte_batch(data, batch_size, context, rng):
233         starts = rng.integers(0, len(data) - context - 1, size=batch_size)
234         windows = torch.stack([
235             data[int(s): int(s) + context + 1] for s in starts
236         ])
237         return windows[:, :-1], windows[:, 1:]
238
239
240     def train_byte_file(
241         path,
242         steps=1000,
243         context=512,
244         batch_size=4,
245         seed=11,
246         device="cpu",
247         save_path="qkra_fixed_window.pt",
248     ):
249         """Small next-byte training loop. Normal language training uses k=0."""
250         torch.manual_seed(seed)
251         rng = np.random.default_rng(seed + 100000)
252
253         raw = Path(path).read_bytes()
254         if len(raw) < context + 2:
255             raise ValueError("training file is too small for the selected context")
256
257         data = torch.from_numpy(np.frombuffer(raw, dtype=np.uint8).astype(np.int64))
258         model = FixedWindowQKLanguageModel().to(device)
259         optimizer = torch.optim.AdamW(model.parameters(), lr=5e-4, weight_decay=0.01)
260
261         model.train()
262         for step in range(1, int(steps) + 1):
263             x, y = make_byte_batch(data, batch_size, context, rng)
264             x, y = x.to(device), y.to(device)
265
266             optimizer.zero_grad(set_to_none=True)
267             logits = model(x, extra_hops=0)
268             loss = F.cross_entropy(logits.reshape(-1, 256), y.reshape(-1))
269             loss.backward()
270             nn.utils.clip_grad_norm_(model.parameters(), 1.0, error_if_nonfinite=True)
271             optimizer.step()
272
273             if step == 1 or step % 100 == 0 or step == steps:
274                 print(f"step={step:5d} loss={loss.item():.4f}")
275
276         torch.save(model.state_dict(), save_path)
277         print(f"saved {save_path}")
278         return model
279
280
281     def smoke_test():
282         torch.manual_seed(123)
283         model = FixedWindowQKLanguageModel()
284         assert model.window == 96
285         assert parameter_count(model) == 271296, parameter_count(model)

```

```

286
287 # 1) Local operator matches the equivalent dense windowed mathematics.
288 h = torch.randn(2, 137, 96)
289 q = torch.randn_like(h)
290 k = torch.randn_like(h)
291 local = local_qk_read(h, q, k, window=96)
292 dense, _ = dense_local_reference(h, q, k, window=96)
293 local_err = float((local - dense).abs().max())
294 assert local_err < 3e-6, local_err
295
296 # 2) Normal and hopped model passes are finite.
297 x = torch.randint(0, 256, (2, 128))
298 y0 = model(x, extra_hops=0)
299 y3 = model(x, extra_hops=3)
300 assert y0.shape == (2, 128, 256)
301 assert y3.shape == (2, 128, 256)
302 assert torch.isfinite(y0).all() and torch.isfinite(y3).all()
303
304 # 3) Hop field is row-normalized and strictly no-self after row zero.
305 final = model.blocks[-1]
306 h2 = final.ln1(
307     model.blocks[0](
308         model.embed(x) * math.sqrt(model.d_model) + model.position[: x.shape[1]],
309         extra_hops=0,
310     )
311 )
312 Ah = strict_hop_matrix(h2, final.qh(h2), final.kh(h2))
313 assert torch.allclose(Ah.sum(-1), torch.ones_like(Ah.sum(-1)), atol=1e-6)
314 diag = torch.diagonal(Ah, dim1=-2, dim2=-1)
315 assert torch.all(diag[:, 1:] == 0), "strict hop rows must not self-loop"
316
317 # 4) Hop parameters are dormant at k=0 and active when hopping is requested.
318 model.zero_grad(set_to_none=True)
319 model(x, extra_hops=0).sum().backward()
320 assert final.qh.weight.grad is None and final.kh.weight.grad is None
321
322 model.zero_grad(set_to_none=True)
323 model(x, extra_hops=1).sum().backward()
324 assert final.qh.weight.grad is not None and torch.isfinite(final.qh.weight.grad).all()
325 assert final.kh.weight.grad is not None and torch.isfinite(final.kh.weight.grad).all()
326
327 print("smoke test: PASS")
328 print("parameters:", parameter_count(model))
329 print("d_model:", model.d_model)
330 print("window W=c*d:", model.window)
331 print("local-vs-dense max abs:", f"{local_err:.3e}")
332 print("normal logits:", tuple(y0.shape))
333 print("+3-hop logits:", tuple(y3.shape))
334
335
336 def main():
337     parser = argparse.ArgumentParser()
338     parser.add_argument("--smoke", action="store_true", help="run architecture checks")
339     parser.add_argument("--train-byte-file", type=str, default=None)
340     parser.add_argument("--steps", type=int, default=1000)
341     parser.add_argument("--context", type=int, default=512)
342     parser.add_argument("--batch", type=int, default=4)
343     parser.add_argument("--seed", type=int, default=11)
344     parser.add_argument("--device", type=str, default="cpu")
345     parser.add_argument("--save", type=str, default="qkra_fixed_window.pt")
346     args = parser.parse_args()
347
348     if args.smoke or args.train_byte_file is None:
349         smoke_test()
350
351     if args.train_byte_file:
352         train_byte_file(
353             args.train_byte_file,
354             steps=args.steps,
355             context=args.context,
356             batch_size=args.batch,
357             seed=args.seed,
358             device=args.device,
359             save_path=args.save,
360         )
361
362
363 if __name__ == "__main__":
364     main()

```

Licensing

Documentation, explanatory text, figures and tables in this quickstart are licensed under the Creative Commons Attribution-NonCommercial 4.0 International licence (CC BY-NC 4.0), subject to attribution and the noncommercial-use restriction.

The accompanying Python source code is licensed separately under the PolyForm Noncommercial License 1.0.0. Neither licence grants unrestricted commercial-use rights. For commercial licensing enquiries, contact rickeyernest1990@gmail.com.